Stoquify moat assessment — execution prompt

Published 27 September 2026. Supplied prompt wording preserved; headings and spacing normalized for publication. Publication destination: `docs/moat/`; the requested dated assessment copy is also retained under `what-next/`.

Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses not applicable with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Maintain explicit review ownership across platform architecture; backend and integrations; data and migrations; security and privacy; frontend engineering; workflow design, accessibility, and localization; product and business processes; quality and release assurance; reliability and operations; and SaaS packaging, growth, and customer success. Activate domain specialists wherever the opportunities touch accounting, compliance, payments, inventory, purchasing, payroll, analytics, or AI.

Mission

Evaluate the Stoquify platform in E:\ohada saas\Focused projects\stoquify and identify the small number of absent or materially incomplete capabilities with the greatest potential to create enduring competitive advantage.

Answer this central question:

Which missing capabilities could become so valuable to Stoquify’s target customers—and so difficult for competitors to replicate effectively—that leaving them unaddressed would materially weaken the platform’s differentiation, adoption, retention, or long-term commercial position?

Treat this as an evidence-led product strategy and capability-gap assessment. Produce recommendations and a validation roadmap; do not implement features.

Execution checklist

1. Establish the platform's actual position. Identify its intended customers, buyers, operating countries, critical workflows, existing strengths, and commercial model. Distinguish documented positioning from assumptions. Assess whether current capabilities already form a defensible advantage that needs completion or better distribution.
2. Verify the gaps. Trace relevant capabilities through routes, UI, actions, services, schema, integrations, permissions, and tests. Classify each as implemented, partially implemented, planned only, not found after bounded inspection, or uncertain. A missing screen does not prove a missing capability, and a report or test name does not prove working behavior.
3. Research the competitive landscape. Select relevant direct competitors, regional alternatives, and substitutes based on the confirmed customer segment. Include spreadsheets, accountants, and disconnected tools where they are realistic alternatives. Use current primary sources, with links and access dates. Distinguish verified competitor functionality from marketing claims and unknowns.
4. Develop and challenge candidate opportunities. Separate essential market expectations, useful differentiators, and capabilities with a credible path to defensibility. Consider whether existing features could create more value through better integration, reliability, usability, or adoption before proposing new modules.
5. Rank the strongest opportunities. Recommend up to five, with fewer if the evidence warrants it. Select one lead investment or validation priority and explain why it outranks the alternatives. Explicitly identify attractive ideas that should be deferred or rejected.

Evidence to inspect

Start with repository instructions, current Git status, project documentation, and relevant material in:

- what-next/ and innovation/ for prior assessments, decisions, roadmaps, and unresolved gaps.
- graphify-out/ for architectural relationships; cite relevant graph nodes or communities and verify material relationships against current source.
- app/, actions/, services/, components/, hooks/, lib/, and config/.
- prisma/, integration boundaries, module entitlements, focused tests, and operational scripts.
- Available customer research, support records, adoption metrics, and commercial evidence that the workspace legitimately contains. Treat historical reports and knowledge graphs as discovery aids, not current implementation proof. Do not infer customer demand or production readiness from code alone.

Required analysis for each shortlisted opportunity

Provide a concise opportunity brief covering:

- Customer and problem: Who uses it, who pays, when the problem occurs, how it is handled today, and the measurable consequence of leaving it unresolved.
- Verified platform gap: What already exists, what is missing, supporting file references, inspection limits, and confidence.
- Proposed experience: One concrete end-to-end customer workflow showing the useful outcome.
- Competitive distinction: Which alternatives address the problem, where Stoquify could credibly outperform them, and supporting sources.

- Defensibility: Explain the causal chain from capability to repeated use, accumulated assets or relationships, improved customer outcomes, and increasing replication difficulty. Specify what could compound: permissioned data quality, workflow integration, expert-reviewed local knowledge, partner distribution, ecosystem participation, or another defensible asset.
- Replication challenge: What could a well-funded competitor copy within six months? What would remain difficult, and why? Identify prerequisites for any claimed network effect.
- Commercial value: Expected effect on acquisition, activation, retention, expansion, willingness to pay, or delivery cost. Label hypotheses and avoid invented revenue or market-size figures.
- Delivery feasibility: Existing foundations, missing dependencies, integrations, operational responsibilities, cost drivers, and a useful first release. Distinguish software delivery effort from partnership, data, or expert-review work.
- Validation: The cheapest credible experiment, target participants, success metric, proposed decision threshold, and evidence that would cause you to reject the idea.
- Cost of omission: Explain specifically what Stoquify risks losing by delaying or omitting this capability, including whether that risk is urgent or speculative.

Do not equate a large feature, an AI assistant, a dashboard, or regulatory complexity with a moat. Demonstrate the mechanism and its supporting evidence.

Prioritization

Use a transparent rubric covering customer pain, recurrence, strategic fit, competitive whitespace, compounding defensibility, adoption friction, monetization potential, and delivery and operating cost.

Keep evidence confidence separate from opportunity scores. Explain the weights and test whether reasonable changes would alter the top recommendation. Prefer ranges and explicit assumptions over false precision.

Distinguish:

- Build or complete now: Strong evidence and sufficient foundations.
- Validate next: Promising value with unresolved demand or feasibility.
- Defer or reject: Weak differentiation, excessive dependencies, poor economics, or inadequate evidence.

Expected artifacts

Save the assessment to what-next/strategic-moat-opportunities-YYYY-MM-DD.md, using the execution date. Include:

1. A direct executive answer naming the strongest opportunities and the lead priority.
2. An evidence-backed inventory of relevant existing capabilities and gaps.
3. A sourced competitor comparison with unknowns clearly marked.
4. Ranked opportunity briefs and rejected alternatives.
5. A dependency-aware validation and delivery sequence, with resource assumptions.
6. An evidence register, unresolved questions, and review disagreements. End with a clear investment recommendation: what to pursue first, what evidence is needed before substantial investment, and what should wait.

Verification commands and checks

Use read-only discovery commands such as git status --short, rg --files, and focused rg -n searches. Record search scope when claiming a capability was not found. Inspect package.json before selecting any verification scripts. Run focused existing checks only when necessary to substantiate a material implementation claim; record passed, failed, skipped, or blocked results. Do not run broad builds or test suites merely to produce a strategy report. Verify that every leading recommendation has repository evidence, dated competitor sources, explicit demand assumptions, and a falsifiable validation plan.

Risk controls

Preserve unrelated working-tree changes and keep sensitive customer, employee, financial, and credential data out of the report. Assess tenant isolation, permissions, entitlements, auditability, human approvals, data rights, and country-specific obligations where an opportunity requires them. Keep business truth server-owned and identify required service contracts before proposing UI surfaces. Treat portability and customer trust as design requirements; do not rely on obstructive lock-in as defensibility.

Success criteria

The assessment is complete when it establishes which gaps are supported by evidence, distinguishes customer value from defensibility, identifies the strongest investment priority, and gives a practical way to validate it before committing substantial resources.

It must be useful for an actual product investment decision, with uncertainty visible and no unsupported claims of indispensability.

Non-goals

Do not modify application code, schemas, dependencies, or production systems. Do not perform unrelated refactoring or lint cleanup. Do not produce a generic feature wish list or assume that every proposed capability deserves implementation.

Optional next prompts

- “Turn the highest-ranked opportunity into a customer-validation brief with interview questions, a prototype scope, measurable success thresholds, and explicit rejection criteria.”
- “After validation, turn the selected opportunity into an implementation brief grounded in existing services, data contracts, permissions, dependencies, and focused acceptance tests.”